ATLAS — Network Service Orchestration Platform

Technical brief prepared for the DriveNets "Senior Python Engineer — Network Orchestration" review

Author Damian Dąbroś · CCIE R&S #40482 **Repository** github.com/damiandabros-lang/atlas (public, MIT) **Date** 19 October 2026

ATLAS provisions, verifies, reconciles and decommissions **MPLS L3VPN services across a service-provider core**. An operator submits a request through a REST API; a durable **Temporal** workflow allocates resources, renders vendor-specific configuration, pushes it to each PE router, verifies BGP/VRF/interface state and connectivity, and commits the service — or, when a device fails part-way, runs a **Saga-style compensation** that restores every touched router and releases the resources. It was designed and built as a production-shaped system (API, database, workflow engine, adapters, observability, Kubernetes packaging, CI), not as a script.

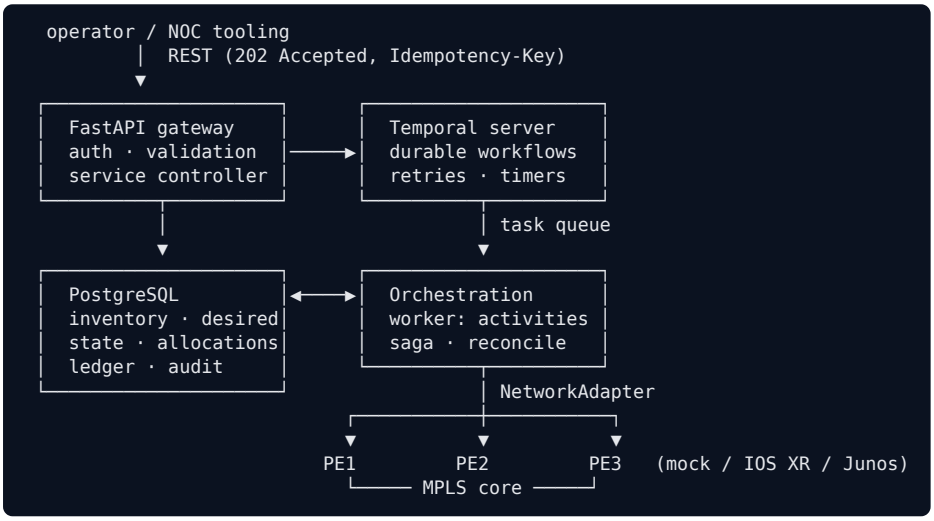
13.6k	174	7 / 7	20	13
lines of Python (incl. 3.9k of tests)	tests: unit + end-to-end, 84 % coverage	CI jobs green on `main`	Temporal activities, 3 workflows	REST endpoints + live topology UI

1. Why this is relevant to the role

Role requirement	What ATLAS demonstrates	Status
Python backend engineering (async, typed, tested)	Python 3.12, FastAPI + Pydantic v2, SQLAlchemy 2 async, structlog; `ruff` + `mypy` clean; 174 tests incl. end-to-end through the real API and workflow engine	Done
Complex workflows / distributed systems	Temporal as the authoritative execution engine: durable execution across worker restarts, per-activity retry policies with exponential backoff, heartbeats, cancellation (signal + native), workflow versioning (`workflow.patched`), determinism rules; Saga compensation in reverse order with an idempotent per-device operation ledger	Done
BGP / MPLS service orchestration	L3VPN domain model (VRF, RD, RT import/export, PE–CE sub-interfaces, static / eBGP PE–CE), vendor templates for IOS XR and Junos, pre-deployment validation, post-deployment verification (VRF/RT/RD, interface, BGP session) and pairwise connectivity matrix	Done
Kubernetes, hands-on	Helm chart: API + worker Deployments with startup/readiness/liveness probes, pre-install/upgrade **migration hook Job** (Alembic + seed), ConfigMap/Secret split with `existingSecret` for ESO/Vault, HPA, PDB, NetworkPolicy, ServiceMonitor, non-root / read-only-FS security context, graceful worker drain (120 s). In-cluster dev dependencies (PostgreSQL, Temporal) for a laptop cluster (OrbStack). `helm lint --strict` + template + kubeconform in CI	Done
PostgreSQL / data integrity	10 tables, FKs, indexes; **concurrency-safe resource allocation** (RD/RT/VLAN/subnet) enforced by a UNIQUE constraint + SAVEPOINT retry rather than locks; optimistic locking (`version`); audit log; Alembic migrations with upgrade/downgrade round-trip in CI	Done
Failure recovery / transactional consistency	Failure classification (transient / permanent / unknown) drives retry vs. compensate vs. inspect-device-state; `MANUAL_INTERVENTION_REQUIRED` when recovery cannot be proven safe; a real cancellation race (write completed while the activity was being cancelled) was caught by the e2e suite and fixed	Done
Observability	Prometheus metrics (`atlas_workflow_*` , `atlas_device_operation_*` , drift), structured JSON logs with secret redaction and correlation	Done

	IDs, OpenTelemetry tracing (FastAPI + Temporal interceptor), Grafana dashboard	
Network automation interfaces	Adapter abstraction (`NetworkAdapter` protocol); stateful mock with failure injection; Cisco IOS XR and Juniper Junos NETCONF adapters (candidate/commit, rollback by commit id, config diff) — ready to point at containerlab / XRd	Done (NETCONF parsers simplified)

2. Architecture



Key design decisions (ADRs in repo)

- **Temporal owns execution state, PostgreSQL owns business state.** No workflow state in the DB, no business data in workflow history.
- **Saga over distributed transactions.** Three routers are never one atomic commit; every device write is a ledger row with an idempotency key, so retries never double-apply and compensation knows exactly what to undo.
- **Resource uniqueness is a data invariant** — enforced by the database, not by a lock or Redis.
- **Reconciliation is DETECT_ONLY** by default; automatic remediation requires explicit authorisation.
- **Adapters are the only place that talks to devices**; swapping mock → NETCONF is configuration.

Provisioning workflow and recovery

```
REQUESTED → VALIDATING → ALLOCATING → (generate configs, pre-deployment validation)
          → PROVISIONING: PE1 ✓ → PE2 ✓ → PE3 ✗ (ConfigurationRejected, permanent)
          → ROLLING_BACK: rollback PE2 ✓ → rollback PE1 ✓ → release RD/RT/VLAN/subnets
          → ROLLED_BACK   (audit: COMPENSATION_EXECUTED ×2, RESOURCES_RELEASED)
Transient failure (TIMEOUT_ON_PE2)  → retried with backoff → … → VERIFYING → ACTIVE
Partial apply (UNKNOWN)             → inspect device state → continue or compensate
Compensation itself fails           → MANUAL_INTERVENTION_REQUIRED (nothing hidden)
```

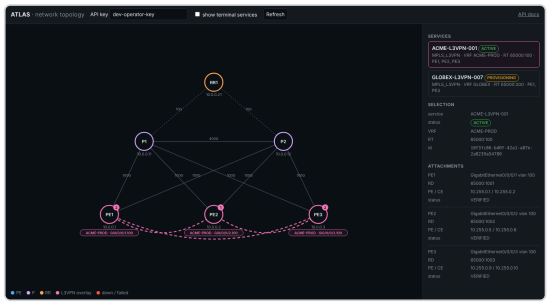
3. Evidence

Test suite (runs in 30 s, no Docker needed)

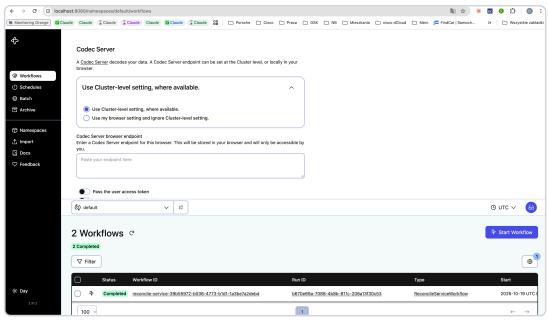
- **Unit** (167): lifecycle state machine, optimistic locking, allocation conflicts and pool exhaustion, adapter failure modes, template rendering, workflow paths with stubbed activities (happy, fail-on-PE3, transient retry, partial apply, cancel before/during, compensation failure), API contract (auth, idempotency replay, 4xx mapping), reconciliation diffs, metrics/logging.
- **End-to-end** (5): real REST API → real Temporal (time-skipping test server) → real activities → database → simulated routers. Asserts on device state, allocations and audit trail. **Found a real bug** during development: a device write completing while its activity was being cancelled was not recorded, so compensation skipped the router. Fixed (`CancelledAfterSettled`).

CI (GitHub Actions, all green)

- ruff lint + format, mypy (clean, 58 files)
- pytest against PostgreSQL 16 + Temporal server services, coverage
- Alembic upgrade → downgrade → upgrade; autogenerate drift check
- multi-arch Docker build → `ghcr.io/damiandabros-lang/atlas`
- Helm lint --strict, template, kubeconform; compose config



Live topology (`/ui/topology`): inventory and core links from the database; selecting a service overlays its VRF attachments and virtual full-mesh. Refreshes every 5 s — provisioning, rollback and decommission are visible as they happen.



Temporal UI: provisioning and reconciliation workflow executions with full event history (activities, retries, timers).

Operational surface

Capability	How
Request a service	`POST /api/v1/services/l3vpn` → 202 + workflow id; `Idempotency-Key` replay; `X-Correlation-ID` through logs, audit and Temporal
Observe	`GET …/status` (lifecycle + live workflow stage), `GET …/history` (audit trail), Temporal UI, Grafana, `/ui/topology`
Operate	`POST …/cancel` (cooperative signal, then native cancel → compensation), `DELETE` (decommission workflow, reverse order), `POST …/reconcile` (desired vs observed per device, e.g. `route_target 65000:100 → 65000:200 = DRIFT_DETECTED`)
Deploy	`docker compose up` (Postgres, Temporal + UI, API, worker, Prometheus, Grafana, optional OTel + Jaeger) · `helm install` with `values-orbstack.yaml` or an external PostgreSQL / Temporal Cloud
Fail safely	`ATLAS_MOCK_FAILURE_MODE=FAIL_ON_PE3|TIMEOUT_ON_PE2|PARTIAL_CONFIGURATION|BGP_SESSION_DOWN|DEVICE_UNREACHABLE` — each with a scripted demo and an e2e test

4. What I would do next (and can discuss in depth)

- Point the IOS XR adapter at containerlab / XRd and replace the mock in the e2e suite with real NETCONF (candidate + commit-confirmed, rollback by commit id).
- Second service type (EVPN / L2VPN) to prove the domain model and workflow generalise; Temporal child workflows per device for parallel provisioning with per-site failure domains.
- AUTO_REMEDIATE behind an explicit approval signal; OIDC instead of API keys; Temporal Cloud + managed PostgreSQL values for production.
- Telemetry-driven verification (gNMI subscriptions) instead of polling, feeding the same reconciliation engine.

Built with Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2, Alembic, PostgreSQL 16, Temporal Python SDK 1.34, Jinja2, structlog, Prometheus, OpenTelemetry, Docker, Helm, GitHub Actions. Developed with AI-assisted tooling under my architecture and review; every design decision, failure-handling rule and test scenario in this document is mine to defend in a technical interview. Repository: github.com/damiandabros-lang/atlas · docs/ARCHITECTURE.md · docs/adr/ · docs/RUNBOOK.md